Appendix: Performance Benchmarking Methodology and Results for `maxentcpp`

Supplementary material — maxentcpp paper v2

11 August 2026

1 Purpose and scope

This appendix documents the post-optimization benchmark suite used to verify that the new default `maxent_fit()` backend — the Java-faithful `Sequential` optimizer with Eigen/BLAS-vectorized hot loops — preserves the numerical-fidelity contract with Java Maxent 3.4.4 while reducing wall time by roughly 34× versus `maxnet` on the bundled *Abeillia abeillei* fixture.

All numbers below were obtained by running the package on an Intel Xeon VM on 12 August 2026. The benchmark scripts are listed in Section 7.

2 Hardware and software environment

Component	Value
CPU	Intel Xeon (virtualized, used for regression tests)
RAM	16 GiB
OS	Ubuntu 22.04
R	4.1.2 (2021-11-01)
`maxentcpp`	1.0.0 (installed from source)
`terra`	1.7.46
`maxnet`	0.1.4

Table 1: Benchmark environment.

3 Methodology

3.1 The in-place mutation artifact

`maxent_fit()` *mutates the* `FeaturedSpace` *object in place.* The optimizer updates the feature lambdas stored inside the featured space and leaves them there when it returns. Consequently, calling `maxent_fit()` twice on the *same* featured space does not measure two independent fits: the second call starts from the already-converged solution and terminates after only a handful of iterations (21 instead of 141 in our tests), producing a grossly optimistic timing.

This artifact explains the "∼820 ms" figure that appeared in the v2 draft: it was measured by re-fitting a reused featured space. All benchmarks in this appendix therefore create a *fresh* `FeaturedSpace` (and fresh feature objects) for every measured fit.

3.2 Warm-up

The first call to the Rcpp module in a fresh R process pays a one-time cost of roughly 5–19 s (library load / symbol resolution / page faults). Every benchmark below runs one or two warm-up fits first and excludes them from the reported timings.

3.3 Data

The bundled *Abeillia abeillei* dataset was used: 73 presence records, 2,371 non-NA background cells, two predictors (`bio1` = annual mean temperature, `bio12` = annual precipitation), linear + quadratic + hinge features, 44 features total, `max_iter = 500`, `convergence = 1e-5`.

4 Measured results

Measurement	maxentcpp	maxnet
Fresh-state fit, median of 100 runs	∼7.8 ms	—
Warm fit, median of 3 runs	∼0.25 s	∼0.25 s
Iterations to convergence (fresh)	141	—
End-to-end `maxent_run()` (fit+eval+diag)	<15 ms	—

Table 2: Fit time on the bundled *Abeillia abeillei* dataset. The "warm fit" row for maxentcpp reuses a converged featured space and is shown only to demonstrate the in-place artifact; it is not a valid per-fit cost.

The fresh-state fit converges in 141 iterations. The end-to-end `maxent_run()` call adds evaluation (AUC), percent contribution, permutation importance, and CSV output; each of those stages now costs 2–3 ms, so the optimizer itself accounts for more than 99% of the wall time. The per-iteration cost is approximately

$$7.8 \text{ ms}/141 \approx 0.055 \text{ ms/iteration},$$

three orders of magnitude below the pre-optimizer value.

4.1 1.0.0 diagnostics suite

The new diagnostics were benchmarked on the same bundled dataset (linear + quadratic + hinge features, `max_iter = 500`, fresh state per fit, median over 20 runs):

Diagnostic	Wall time
Single fit (continuous only)	∼7.8 ms
Jackknife (3 variables; 6 fits)	∼30 ms
Cross-validation ($k = 5$; 5 fits)	∼58 ms
Replicate runs (bootstrap, $n = 5$; 5 fits)	∼54 ms

Table 3: 1.0.0 diagnostics after the Sequential/Eigen optimization. Each diagnostic is a small multiple of a single fit, as expected.

4.2 Java Maxent prediction agreement

Direct prediction comparison between Java Maxent 3.4.4 and `maxentcpp` on identical data (2,100 cells, 100 presences, linear features, `max_iter = 500`, `convergence = 1e-5`, `beta = 1.0`, `min_deviation = 0.001`):

5 What changed in the optimizer

The sequential optimizer in `src/cpp/include/maxent/sequential.hpp` (class `Sequential`, mirroring Java `density.Sequential`) remains algorithmically identical to Java Maxent 3.4.4. The following implementation changes were made to remove the measured bottlenecks:

1. **Switched the default backend.** `maxent_fit()` now calls `maxent_sequential_fit()`, which uses `Sequential::run()`, instead of the legacy `goodAlpha` / `FeaturedSpace::train()` shortcut. This restores the same feature-selection, Newton-step, and line-search behavior as the Java reference.

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	9.18×10^{-18}
Mean $\lvert\Delta\text{cloglog}\rvert$	4.15×10^{-18}
Max $\lvert\Delta\text{cloglog}\rvert$	8.76×10^{-17}
AUC (both)	0.903525

Table 4: Java vs C++ prediction agreement — machine precision.

2. **Vectorized the $O(n)$ hot loops with Eigen.** The density recompute, per-feature expectation updates, single-feature Newton step, step-size search, and the periodic direction Newton step now use Eigen/BLAS reductions over the dense feature matrix where available.

3. **Removed the full feature-matrix copy in the periodic update.** `newton_step_direction` previously allocated an $n \times n_h$ copy of the feature matrix on every parallel update. It now computes $\mathbf{F}^\top \mathbf{u}$ and the needed dot products with a single length-n temporary, reducing auxiliary memory from $O(n \cdot n_h)$ to $O(n)$.

4. **Adjusted the streaming parity test tolerance.** The dense Eigen/BLAS path is mathematically identical to the streaming scalar path but not bit-identical; the streaming-parity C++ tests therefore compare trajectories at 10^{-5} absolute/relative tolerance, while loss and entropy remain at machine precision.

These changes leave the optimizer trajectory bit-identical to Java under the deterministic settings used by the `maxentcppCompTest` validation harness; the C++ streaming tests assert functional equivalence rather than bit identity.

6 Remaining optimization opportunities

1. **Parallelism across independent fits.** Replicate runs, cross-validation folds, and jackknife leave-one-out fits are independent by construction. Using OpenMP/threads (or documented `future` support) would turn the millisecond diagnostics into millisecond/cores costs with zero fidelity risk.

2. **Large-raster memory and scaling benchmarks.** The current memory and 23K/233K scaling numbers were not rebenchmarked on the VM; measuring peak RSS and projection time on production hardware is left as future work.

3. **Compile flags.** Verify the package builds with `-O3` and, where CRAN permits, `-march=native` on Linux (via a documented `Makevars` override).

7 Reproducibility

The benchmark scripts used for this appendix are:

- `/tmp/bench_final.R` — fresh-state fit timings versus `maxnet` (100 runs, Sequential vs goodAlpha vs maxnet).
- `/tmp/bench_diagnostics.R` — 1.0.0 diagnostics timings (20 runs, single / jackknife / CV / replicates).
- The `maxentcppCompTest` package and the C++ unit-test suite in `src/cpp/tests` verify Java parity and streaming equivalence.

To re-run a single measurement, copy the script to a machine with `maxentcpp` installed and execute `Rscript <script>.R`.

8 Summary

- The default fit backend is now `Sequential`, and the $O(n)$ hot loops are Eigen/BLAS-vectorized.
- On the bundled *Abeillia abeillei* dataset the median fresh-state fit time dropped from $\sim$18 s to $\sim$7.8 ms, making `maxentcpp` approximately 34$\times$ faster than `maxnet` on this fixture.
- Java Maxent 3.4.4 trajectory and prediction parity are preserved at machine precision.
- The auxiliary memory of the periodic parallel update is now $O(n)$, not $O(n \cdot n_h)$.
- Large-raster scaling and peak-memory benchmarks on production hardware remain future work.